

Predictive Maintenance Without Labels

How the model learns 'maintenance is required' from ordinary telemetry — and keeps learning from operators while it runs

A complete walk-through of a label-free predictive-maintenance model built as a Jneopallium neuron network: what each neuron does, why it needs no fault history, how it keeps improving from operator feedback without a redeploy, and why it stays strictly advisory.

Document	Architecture Article
Product	Jneopallium Self-Supervised Maintenance Guardian
Module	self-supervised-maintenance 1.0.0-label-free-continuous
Runtime	Java neurons · net.neuron.impl.ssmaint
Training	Python (initial fit only) · standard library
Safety	ADVISORY / read-only — never actuates
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	2 July 2026
License	BSD 3-Clause

Contents

1. The problem in one paragraph
2. For the non-specialist: the whole idea
3. What the model observes
4. The five layers at a glance
5. Layer 1 — self-supervised reconstruction
6. Layer 2 — the maintenance hypothesis
7. Layer 3 — continuous learning from feedback
8. Layer 4 — the read-only advisory gate
9. The signals that connect it all
10. Why no labels are needed
11. How it keeps learning without a redeploy
12. Safety model
13. What it cannot do (honest limits)
14. Deployment shape

1 The problem in one paragraph

Most predictive-maintenance models are supervised: you feed them a history of past failures and repairs, and they learn to recognise the run-up to each one. That works — once you have a labelled failure history. Most sites do not. They have years of sensor telemetry and a maintenance log that is patchy, free-text, or simply absent. The Self-Supervised Maintenance Guardian is the model for exactly that situation: it learns what 'healthy' looks like directly from the telemetry, flags departures from it as developing faults with a lead time, and sharpens itself over time from the one form of supervision that arrives for free — an operator saying 'yes, that was real' or 'no, that was a false alarm.'

2 For the non-specialist: the whole idea

Imagine a seasoned operator who has listened to a pump for years. Nobody handed them a textbook of labelled failures; they simply know what the machine sounds like on a good day, and they notice when something drifts. When it does, they don't shout 'FAILURE' — they say 'keep an eye on bearing 2, it's warming up.' If they're wrong a few times, they recalibrate their own sense of 'normal.' This model is that operator, written down.

- It learns 'normal' from the data itself — no labelled failures required.
- It watches several sensors together, because a real fault shows up as a pattern across them, not a single needle in the red.
- It only raises a flag when the drift is persistent, trending, and consistent — not on a one-second blip.
- It tells you which kind of problem it looks like and roughly how long you have.
- When an operator confirms or dismisses a flag, it quietly adjusts — while it keeps running.
- It never touches the machine. It only ever advises a human.

One sentence to remember

The model's targets are the sensors themselves, not a failure label — which is why it can be trained on nothing but ordinary operating data.

3 What the model observes

The only input is a telemetry frame — one reading of each sensor at one moment, tagged with the current operating regime (low / medium / high production load). The reference build uses eight coupled pump-skid channels:

Channel	What it is	Why it matters
suction_pressure	Pump inlet pressure	Falls in cavitation
flow	Process flow rate	Sets the operating point; unstable in cavitation
pump_speed	Shaft speed	Normalises everything else
pump_power	Motor power draw	Rises with wear and with lost efficiency
vibration_rms	Vibration level	Rises with bearing wear and cavitation

bearing_temp	Bearing temperature	Rises with bearing wear
process_temp	Process temperature	Drifts when its sensor fails
valve_position	Control valve travel	Oscillates with loop/stiction problems

The channels are physically coupled — power tracks flow, bearing temperature tracks vibration — and that coupling is exactly what the model exploits. The regime tag lets it judge a reading relative to its load, so a busy shift does not look like a fault.

4 The five layers at a glance

Layer	Neuron	Turns ... into ...
0	(telemetry ingest)	sensor frame → AssetTelemetrySignal
1	CrossSensorReconstructionNeuron	telemetry → reconstruction residual
2	MaintenanceHypothesisNeuron	residual → maintenance hypothesis + lead time
3	FeedbackAdaptationNeuron	operator feedback → threshold update
4	SsAdvisoryGateNeuron	hypothesis (+ update) → read-only advisory

Layers 1–2 and 4 form the fast detection path (telemetry in, advisory out). Layer 3 is the slow learning path (feedback in, thresholds out) that feeds back into layer 4. Each neuron is single-purpose; capacity is added as layers, not as tangled logic inside one neuron.

5 Layer 1 — self-supervised reconstruction

This is the heart of the label-free idea. For each sensor the neuron holds a small linear model — fitted offline on a trusted-healthy window — that predicts that sensor from all the others, after standardising by regime. At runtime it compares prediction to reality:

```
residual_i = actual_i - predicted_from_peers_i    (standardised)
total      = mean over sensors of residual_i2
```

A large **total** means the joint pattern no longer looks healthy. A large **single** residual means one sensor disagrees with everyone else — the classic signature of a failing sensor. Because each model's target is another sensor, no failure label is ever needed; the data supervises itself. The neuron also reports a **domain-shift** fraction: how many sensors sit far outside the envelope it was trained on, i.e. how unfamiliar this reading is.

Plain language

The neuron asks each sensor: 'given what your neighbours are doing, what should you read?' When a sensor's real value drifts away from what its neighbours imply, that gap is the early warning.

6 Layer 2 — the maintenance hypothesis

One residual is noise; a maintenance case is a story over time. The hypothesis neuron keeps, per asset, four running quantities and fuses them:

- **Trend** — a slow moving average of the residual and its slope, so a steady climb is distinguished from a spike.
- **Change-point** — a Page-Hinkley detector that fires when behaviour steps to a new level.
- **Severity** — the level expressed in the asset's own baseline-percentile units, so '1.0' means 'at the 99th percentile this asset has ever shown.' Calibrated from healthy history, not from labels.
- **Evidence** — an accumulator that only rises when the deviation is severe, trending, consistent across at least two sensors, and not merely a domain shift.

It attributes a **fault family** heuristically from which sensors dominate the residual (bearing, cavitation, sensor, energy, oscillation, or an honest 'unknown'), and it extrapolates the trend to the baseline limit to estimate a **lead time** — the label-free stand-in for 'how long until this needs attention.' Uncertainty rises with domain shift, so an unfamiliar asset reads as 'watching, not sure' instead of a confident wrong answer.

7 Layer 3 — continuous learning from feedback

When an operator marks an advisory confirmed or false-positive, that verdict arrives as an `OperatorFeedbackSignal`. The adaptation neuron holds a per-fault-family offset on top of a static base threshold and moves it:

- A **false positive** raises that family's threshold — fewer nuisance alerts of that kind.
- A **confirmed need** relaxes it slightly — recall is already high, so the nudge is small and deliberately asymmetric.

Every update is **bounded** (the offset is clamped), **rate-limited** (a minimum interval between changes), and **frozen during domain shift** so a novel or abnormal period cannot poison the thresholds. The neuron owns this state, which the runtime persists to storage — so the learning survives a restart. Crucially, none of this requires rebuilding or redeploying the model: it emits a `ThresholdUpdateSignal` that the gate applies in place.

8 Layer 4 — the read-only advisory gate

The terminal stage turns a hypothesis into a human advisory when the accumulated evidence clears the family's live threshold, subject to a per-asset de-duplication window so one developing fault is not reported every tick. It holds its thresholds in a live map that the feedback path mutates in place — which is how the operators' verdicts reshape gate behaviour while the process keeps running. The only thing it can emit is a `MaintenanceAdvisorySignal`; it has no path to an actuator, by construction.

Regression guard worth noting

The dedup window and the feedback rate-limit originally used a sentinel that overflowed on real epoch-millisecond timestamps, which would have suppressed the very first advisory and the first feedback in production. Both are fixed and covered by tests.

9 The signals that connect it all

Signal	Carries	Cadence (epoch/loop)
AssetTelemetrySignal	one multi-sensor frame + regime	1 / 1 (fast)
ReconResidualSignal	total + per-sensor residuals + domain shift	1 / 1
HealthHypothesisSignal	family, severity, evidence, lead time, uncertainty	2 / 2
OperatorFeedbackSignal	confirmed / false-positive + provenance	10 / 2 (slow)
ThresholdUpdateSignal	family threshold + offset	10 / 2
MaintenanceAdvisorySignal	read-only recommendation + lead time	2 / 2

Every signal is a typed ISignal with its own ProcessingFrequency, so the fast detection path and the slow learning path are scheduled independently on the same engine.

10 Why no labels are needed

Each learning objective is self-supervised — its target is something the model can check against reality without a human: reconstruct a sensor from its peers, calibrate severity against the asset's own history, extrapolate a trend to a limit. None of these require knowing that a specific past event 'was a bearing failure.' The consequence is practical: you can deploy on a site that has telemetry but no maintenance-labelled history, which is most sites.

11 How it keeps learning without a redeploy

Learning at runtime lives in neuron state, not in the packaged JAR. The worker runs continuously; the adaptation neuron adjusts thresholds on the slow loop and persists them to the configured storage. A separate champion/challenger discipline is the recommended way to evolve the heavier parameters safely: a stable champion serves advisories while a challenger adapts in shadow and is promoted only when it beats the champion on independent outcomes. Nothing here interrupts the running process.

12 Safety model

- **Advisory-only invariant.** SsMaintConfig forbids disabling the advisory posture; no neuron in the module has a path to an actuator.
- **The safety layer is never learned.** Threshold adaptation touches scores and alert thresholds — never a hard interlock or a safety gate.
- **Honest uncertainty.** Domain-shift and uncertainty are first-class outputs, so the model can decline to claim confidence on an unfamiliar asset.
- **Anti-poisoning.** Adaptation freezes during domain shift; baselines adapt only from trusted-healthy periods.

13 What it cannot do (honest limits)

Label-free mode detects **degradation**, not a contractually-due repair. Fault families are heuristic hypotheses until confirmed events accumulate, so expect more false positives than a fully label-trained model — which is exactly why the feedback loop exists. It also needs a **mostly-healthy** training window (or fleet peers) to define normal; a brand-new asset with no history starts in a high-uncertainty, domain-shift-aware mode rather than making confident claims.

14 Deployment shape

The trained model is a Jneopallium bundle — a model descriptor, an IContext, and per-layer neuron configuration carrying the fitted parameters — loaded by the standard Entry runner in local, HTTP, or gRPC mode. Telemetry arrives through the usual OPC UA / MQTT inputs; advisories leave through an output aggregator (JSONL, Kafka, or MQTT). The companion Deployment Guide covers the full runbook; the Training Guide covers fitting the parameters from your own historian.

Self-Supervised Maintenance Guardian — label-free detection, feedback-driven learning, and a safety posture that never leaves advisory. It is the model for the site that has data but not yet a labelled failure history.